POLITECNICO DI BARI



DIPARTIMENTO DI INGEGNERIA ELETTRICA
E DELL'INFORMAZIONE
CORSO DI LAUREA MAGISTRALE IN INGEGNERIA INFORMATICA

Formal Languages and Compilers Project Assignment Report

OWL Stream Protocol Syntax Parser and Renderer

Docente del corso:
Prof. Floriano Scioscia

Studenti:
Apollo Beatrice, Cassano Mario

Anno Accademico 2025/2026

Sommario

1. Contesto e obiettivi del progetto....3
1.1 ProtocOWL....3
1.2 OWLAPI....3
1.3 Step operativi....4
2. Il codice di partenza....6
2.1 Struttura della codebase....6
3. Aggiornamento delle costanti....8
4. Aggiornamento del Parser....9
4.1 Struttura dello stream e frame....9
4.2 Metadati....9
4.3 Namespace e prefissi....10
4.4 Annotazioni....10
4.5 Literal....10
4.6 Espressioni di classe e cardinalità....11
4.7 Assiomi....12
4.8 Gestione degli errori....13
4.9 Reset e maximum index size....13
5. Aggiornamento del Renderer....15
5.1 Raccolta degli identificativi....15
5.2 Metadati e annotazioni....15
5.3 Namespace e prefissi....15
5.4 Literal....16
5.5 Assiomi....17
5.6 Espressioni di classe e cardinalità....17
5.7 Terminazione dello stream....18
5.8 Gestione dei costrutti non supportati....18
6. Aggiornamento dei test e risultati....19
6.1 Risorse e organizzazione dei test....19
6.2 Criteri di confronto....19
6.3 Test negativi e regressioni mirate....20
6.4 Risultati della verifica....21

6.5 Dimensione dei file....21
6.6 Esecuzione della suite....22
6.7 Copertura dei test....22
6.8 Analisi di copertura sul dataset....23
6.9 Round-trip completo e conservazione strutturale....23
7. Benchmark: metodologia e stato della validazione....25
7.1 Dataset e combinazioni di misura....25
7.2 Misura di tempo e memoria....25
7.3 CSV e validazione delle metriche....26
7.4 Ambiente ed esecuzione riproducibile....26
7.5 Stato della validazione e anomalie nei dati....27
7.6 Risultati della campagna....28
8. Conclusioni e sviluppi futuri....30
8.1 Difficoltà riscontrate....30
8.2 Sviluppi futuri....33
9. Bibliografia....34

Sezione 1

Contesto e obiettivi del progetto

Il progetto, sviluppato nell'ambito del corso di Formal Languages and Compilers, riguarda l'aggiornamento di un parser e di un renderer Java per ProtocOWL, un formato binario destinato allo scambio di ontologie OWL. Il lavoro comprende l'allineamento alla specifica ProtocOWL versione 1.0 [1], il supporto alle ontologie di riferimento home.owl e pizza.owl e una successiva fase di validazione su un sottoinsieme di 100 ontologie ORE 2014 [2]. Quest'ultima fase introduce la lettura della variante con maximum index size pari a 128, l'analisi di copertura, il round-trip completo attraverso OWLAPI e Functional Syntax e un benchmark comparativo di tempi, memoria e dimensioni.

Il nucleo del progetto è la traduzione tra due rappresentazioni della stessa conoscenza: da una parte gli oggetti del modello OWLAPI, dall'altra una sequenza di byte organizzata in frame. La correttezza richiede che questa traduzione conservi gli assiomi e i metadati trattati, rispettando sia la struttura binaria del protocollo sia il significato dei costrutti OWL. Un errore nel codice di un frame, nell'indice di un identificativo o nell'interpretazione di un literal può infatti rendere il documento illeggibile oppure produrre un'ontologia diversa da quella iniziale.

1.1 ProtocOWL

ProtocOWL è un formato binario compatto e un protocollo leggero progettato per lo scambio di conoscenza OWL in forma di stream, con particolare attenzione a reti Machine-to-Machine e dispositivi con risorse limitate [1], [3]. La specifica organizza il contenuto in frame che possono rappresentare dichiarazioni, metadati, assiomi o informazioni di controllo. Le funzioni di indirizzamento, trasporto e gestione degli errori di comunicazione restano affidate agli strati di rete sottostanti.

1.2 OWLAPI

OWLAPI, usata nella versione 5.5.1, è la libreria Java per rappresentare e manipolare le ontologie [4]; il suo modello segue la specifica strutturale di OWL 2, che definisce anche la Functional-Style Syntax [5]. Nel progetto fornisce le classi relative a entità, espressioni, assiomi, literal e annotazioni, oltre ai manager che coordinano caricamento e salvataggio. Il

parser ProtocOWL ricostruisce questi oggetti attraverso una OWLDataFactory; il renderer percorre invece il modello OWLAPI per produrre la rappresentazione binaria corrispondente.

OWLAPI consente anche di verificare il risultato della conversione confrontando direttamente oggetti e insiemi di assiomi, indipendentemente dall'ordine di serializzazione; i criteri del confronto, strutturale e non basato su un reasoner, sono descritti nella sezione 6.2.

1.3 Step operativi

Il primo step ha avuto come obiettivo l'allineamento alla specifica v1 per supportare home.owl. Questa ontologia descrive un ambiente domestico con stanze, dispositivi, sensori e proprietà associate. Le attività hanno riguardato la sostituzione delle risorse di test obsolete, la rimappatura dei codici, la gestione del frame End, l'aggiornamento dei metadati e la lettura e scrittura dei literal compatti. Sono stati inoltre riorganizzati i test per consentire l'utilizzo di più ontologie e introdotti controlli su alcuni input non validi.
Il secondo step ha esteso il supporto a pizza.owl, ontologia di esempio del Pizza Tutorial dell'Università di Manchester [6], che contiene una struttura più articolata di classi, proprietà, restrizioni e annotazioni. Sono state ampliate le espressioni di classe e aggiunti assiomi di disgiunzione, assiomi sulle proprietà oggetto, relazioni tra individui e AnnotationAssertion. Le verifiche hanno richiesto anche di conservare il version IRI, i literal con lingua, i valori IRI delle annotazioni e l'intera mappa dei prefissi.
Il terzo step si articola in tre ulteriori fasi, da svolgere in ordine. La prima fase verifica la copertura dei costrutti sulle 100 ontologie del dataset: per ogni ontologia si leggono entrambe le varianti ProtocOWL e si prova la serializzazione del Functional di riferimento. La seconda fase verifica, separatamente per ciascuna variante, il percorso ProtocOWL -> OWLAPI -> Functional -> OWLAPI -> ProtocOWL standard -> OWLAPI, confrontando ogni risultato con il Functional iniziale. La terza prevede la misura di tempi, memoria e dimensioni e la produzione di un CSV completo di 500 righe dati.

La variante MIS_128 richiede il supporto al frame Reset soltanto in lettura. Il renderer continua a produrre la codifica standard, senza Reset. Le ontologie che falliscono non vengono escluse dal campione: un costrutto mancante richiede un'estensione di parser e

renderer, mentre un'incoerenza dei dati deve essere documentata e risolta prima di considerare conclusa la validazione.

Sezione 2

Il codice di partenza

Il codice iniziale conteneva già l'infrastruttura necessaria a leggere e scrivere documenti ProtocOWL mediante OWLAPI. Il parser manteneva le tabelle di namespace e identificativi, decodificava VarInt e stringhe e costruiva gli oggetti OWL. Il renderer eseguiva il percorso inverso, registrando le entità prima di emettere dichiarazioni e assiomi. Erano disponibili due test, parserTest e rendererTest, riferiti a una sola coppia di documenti test.owl e test.oprt. Il limite principale era l'incompatibilità della tabella dei codici con la nuova specifica. Per esempio, il valore 0x03 indicava nel vecchio codice una dichiarazione di data property, mentre nella nuova versione identifica End. Anche alcuni tag delle espressioni di classe erano diversi: ObjectSomeValuesFrom e ObjectAllValuesFrom utilizzavano rispettivamente 5 e 6, anziché 4 e 5. La presenza della costante di versione uguale a 1 non era quindi sufficiente a garantire la conformità del contenuto binario.

Il supporto iniziale comprendeva i sei tipi di dichiarazione di entità, SubClassOf, EquivalentClasses, ClassAssertion, ObjectPropertyAssertion e DataPropertyAssertion. Le espressioni di classe complesse erano limitate a ObjectIntersectionOf, ObjectSomeValuesFrom e ObjectAllValuesFrom. Erano già previste proprietà oggetto dirette e inverse e la distinzione tra literal semplici, con lingua e tipizzati, ma il valore dei literal era sempre trattato come stringa.

Il version IRI veniva consumato in lettura senza essere conservato e non era scritto dal renderer; mancavano imports e annotazioni dell'ontologia. I frame non riconosciuti erano ignorati dal parser. La raccolta degli identificativi, pur includendo l'IRI dell'ontologia, non copriva gli ulteriori riferimenti necessari per i metadati e le annotazioni di pizza. I due test iniziali passavano perché test.oprt era codificato nel vecchio formato e quindi non verificavano la compatibilità con i file v1 di riferimento.

2.1 Struttura della codebase

Il nucleo del codec è costituito da Constants.java, Parser.java e Renderer.java. Constants.java raccoglie i codici dei frame e dei tipi complessi. Parser.java ricostruisce l'ontologia dallo stream e gestisce i frame Reset; una revisione selezionabile esplicitamente legge la codifica precedente degli interi usata da home.oprt (sezione 4.5). Renderer.java

raccoglie i riferimenti e serializza metadati e assiomi nella codifica standard. Le classi di integrazione con OWLAPI continuano a esporre parser, storer e document format.

La verifica è suddivisa fra più componenti. ProtocOWLTest.java mantiene i casi home e pizza, i controlli di base e il confronto condiviso fra ontologie. BenchmarkComplianceTest.java aggiunge regressioni su assiomi annotati, Reset, riferimenti invalidi e valori numerici. CodecRobustnessTest.java verifica overflow, cardinalità limite e conservazione rigorosa dei prefissi. IntegerEncodingTest.java verifica la codifica e decodifica dei valori numerici, le forme lessicali e input troncati, includendo i casi di fixed point negativi. DatasetRoundTripTest.java enumera le 200 coppie ontologia/variante e verifica il percorso completo a/b/c.

Nella cartella benchmark, DatasetFiles.java centralizza l'elenco delle 100 ontologie e la risoluzione dei percorsi; CoverageAnalyzer.java produce il rapporto di copertura; BenchmarkTask.java esegue una singola misura; BenchmarkResults.java valida il CSV finale, con regressioni in BenchmarkResultsTest.java. run_benchmark.sh coordina prerequisiti, misure in JVM separate e registrazione dell'ambiente. Il file lib/build.gradle.kts separa i test rapidi dai gruppi coverage e dataset.

Le risorse home e pizza restano disponibili per la regressione iniziale. Il dataset della nuova fase è esterno alle risorse di test ed è individuato attraverso un elenco condiviso. DatasetPreparer.java appartiene alla precedente infrastruttura: la pipeline corrente non lo usa per filtrare o rigenerare il campione consegnato.

Sezione 3

Aggiornamento delle costanti

Constants.java è stato aggiornato per allineare i codici alla specifica ProtocOWL v1, distinguendo frame di controllo, dichiarazioni, metadati e assiomi. La revisione comprende anche i tag delle espressioni di classe e le costanti necessarie alla gestione dei literal compatti. Per esempio, NamespaceDeclaration passa da 0x2A a 0x04, SubClassOf da 0x06 a 0x0F e la dichiarazione di data property da 0x03 a 0x0C. È stata inoltre eliminata la costante separata per l'IRI versionato: l'IRI dell'ontologia e l'eventuale version IRI utilizzano lo stesso frame, con un utility bit che segnala la presenza della versione. Per supportare il dataset di ontologie, vengono utilizzati anche i tag delle restrizioni sulle data property, delle cardinalità sui dati e dei data range. Sono disponibili i codici dei formati numerici fixed point ed E-notation, decodificati dal parser. Sono stati inoltre aggiunti i frame AnnotationPropertyDomain e AnnotationPropertyRange, rispettivamente 0x31 e 0x32, con i corrispondenti metodi di lettura e scrittura. Reset (0x02) è gestito in lettura; le costanti di Add (0x00) e Remove (0x01) sono definite, ma il parser rifiuta questi frame come non supportati. I nomi interni LITERAL_FMT_FLOAT e LITERAL_FMT_DOUBLE designano i formati 5 (fixed point) e 6 (E-notation), non i datatype xsd:float e xsd:double: il datatype resta indicato dall'identificativo del literal tipizzato.

Per i literal interi, LITERAL_FMT_POSITIVE_INT identifica il formato 3, riservato ai valori non negativi, incluso zero; LITERAL_FMT_NEGATIVE_INT identifica il formato 4, riservato ai valori negativi. In entrambi i casi il modulo viene codificato come unsigned VarInt e il segno è determinato dal formato. L'uso di SVarInt resta distinto e riguarda le componenti con segno dei formati decimali compatti.

Sezione 4

Aggiornamento del Parser

Le modifiche al parser riguardano la struttura dello stream, i metadati, i costrutti necessari alle ontologie di riferimento e le estensioni introdotte per il dataset ORE. Oltre alla codifica corrente, il parser conserva due percorsi di compatibilità con formati precedenti, descritti nelle sezioni 4.1 e 4.5.

4.1 Struttura dello stream e frame

Il parser legge il byte di versione e procede sequenzialmente sui frame. Per ogni header estrae il tipo con la maschera 0x3F e i due utility bit, cioè i bit 6 e 7 (nel seguito primo e secondo utility bit), mediante uno spostamento di sei posizioni. Nella codifica corrente il frame End interrompe la lettura; rimane ammessa anche la terminazione alla fine dello stream. I tipi non riconosciuti producono un errore esplicito. Per compatibilità con il formato precedente, se il primo frame ha tipo 0x2A, 0x2B o 0x2C (i vecchi codici di NamespaceDeclaration, IdentifierDeclaration e IRI dell'ontologia) il parser usa la vecchia tabella dei codici.

I 200 binari del dataset iniziano con il byte di versione 1 e con un frame NamespaceDeclaration (0x04), quindi usano la tabella dei codici v1. I 100 binari standard terminano alla fine dello stream, senza End, mentre i 100 binari MIS_128 si chiudono con End: il parser accetta entrambe le forme.

Per i frame di assioma della codifica corrente, il primo utility bit segnala la presenza di annotazioni, lette prima del contenuto dell'assioma. Nella sottoproprietà oggetto il secondo bit distingue la catena dalla forma semplice. Questa separazione permette di conservare anche le annotazioni delle catene di proprietà.

4.2 Metadati

La lettura dei metadati comprende l'identità dell'ontologia, la versione, gli imports e le annotazioni. parseOntologyIRI legge l'identificativo dell'ontologia e, quando il primo utility bit è impostato, anche quello della versione. Entrambi sono utilizzati per costruire l'OWLOntologyID, poi applicato tramite SetOntologyID. parseImports legge la sequenza

degli IRI importati e aggiunge le dichiarazioni mediante AddImport. parseAnnotations ricostruisce le annotazioni dell'ontologia e le inserisce mediante AddOntologyAnnotation.

4.3 Namespace e prefissi

La gestione dei namespace mantiene i cinque indici riservati e normalizza i nomi dei prefissi secondo le convenzioni OWLAPI. Gli alias dei namespace non riservati vengono conservati quando sono dichiarati nello stream.

Le tabelle necessarie alla decodifica sono distinte dalla mappa dei prefissi del documento. Un Reset dei namespace elimina le mappature non riservate usate dai riferimenti successivi, ma non cancella i prefissi già raccolti.

4.4 Annotazioni

Il parser distingue annotazioni dell'ontologia, AnnotationAssertion e annotazioni associate agli assiomi. Un valore di annotazione può essere un IRI, un individuo anonimo o un literal. Nella codifica corrente, se il primo utility bit di un frame di assioma è impostato, viene letta la sequenza di annotazioni che precede il contenuto. L'assioma costruito viene poi arricchito con tali annotazioni. Il meccanismo comprende anche declaration e annotation assertion.

Le annotazioni possono a loro volta contenere sottoannotazioni. Nella forma semplice la proprietà è codificata come identificativo aumentato di uno; nella forma annotata il tag zero introduce la sequenza delle sottoannotazioni e il successivo identificativo della proprietà non usa tale incremento. Parser e renderer sono stati corretti per distinguere le due forme.

4.5 Literal

Il parser separa il tipo del literal dal formato binario del valore. Il tipo viene estratto con header & 0x03 e il formato con (header >> 2) & 0x3F. La lettura supporta stringhe, booleani, interi non negativi e negativi e decimali compatti fixed point ed E-notation; il formato 1, riservato alle stringhe compresse, è rifiutato. Il parser accetta anche un tipo 3, non previsto dalla specifica, con datatype dedotto dal formato; nessun file del dataset lo usa. Nei literal tipizzati viene letto anche l'identificativo del datatype: il formato compatto non lo sostituisce. Le stringhe plain vengono ricostruite come literal stringa OWLAPI, mentre i literal con lingua conservano il language tag.

I booleani accettano soltanto i byte 0 e 1. Il formato 3 legge il valore non negativo come unsigned VarInt; il formato 4 legge il modulo come unsigned VarInt e applica il segno negativo. Per esempio, un payload 05 nel formato 4 restituisce -5. Questa è la codifica della revisione corrente della specifica [1]. Nella revisione precedente il formato 3 era un SVarInt con segno e il formato 4 un VarInt senza segno: home.oprt, unico file di riferimento con literal interi, codifica così i valori positivi nel formato 4. Poiché il byte di versione è 1 in entrambe le revisioni, il test legge i binari di riferimento con la revisione LEGACY del parser, selezionata esplicitamente, che segue la codifica precedente anche per i fixed point negativi. I valori numerici compatti vengono accumulati tramite BigInteger; indici, conteggi e lunghezze mantengono invece una funzione distinta con controlli dei limiti prima della conversione a int, descritti nella sezione 4.8.

Nei decimali il protocollo memorizza la parte frazionaria con le cifre invertite. Per ricostruire 31.05 occorre leggere parte intera 31 e frazione 50, invertendo quest'ultima in '05'. La forma E-notation aggiunge un esponente con segno. Nel formato fixed point 5, la parte intera resta codificata come SVarInt e la frazione come unsigned VarInt con le cifre invertite. La specifica non indica come rappresentare la parte intera dei valori negativi: come chiarito dal docente, essa è diminuita di uno prima della codifica. Dopo la decodifica SVarInt, se il risultato s è negativo si ricostruisce la parte intera testuale come '-' seguito da abs(s + 1). Il segno viene conservato separatamente anche quando il modulo è zero: s = -1 produce '-0', necessario per distinguere -0.5 da 0.5. La compensazione riguarda soltanto il formato 5: la funzione zig-zag non cambia e nel formato 6 parte intera ed esponente sono letti senza correzione.

Per i literal tipizzati trasmessi in forma testuale il parser conserva la stringa presente nello stream anche quando la factory OWLAPI la normalizzerebbe. Questo permette, per esempio, di rileggere '+1' senza trasformarlo in '1'. Non si possono invece recuperare informazioni già normalizzate durante il caricamento del documento di origine.

4.6 Espressioni di classe e cardinalità

Il supporto alle espressioni di classe è stato esteso a unioni, complementi, enumerazioni di individui, restrizioni ObjectHasValue e ObjectHasSelf e cardinalità minime, massime ed esatte. Il parser legge ricorsivamente gli operandi delle espressioni annidate e utilizza la data factory per costruire il corrispondente oggetto OWLAPI.

Nel campo delle cardinalità il bit meno significativo segnala la presenza del filler, mentre i bit rimanenti rappresentano il numero richiesto. Il parser ricava quindi la cardinalità spostando il campo a destra di una posizione e legge il filler soltanto quando il relativo bit è impostato. In assenza di filler esplicito utilizza owl:Thing, come previsto per le restrizioni non qualificate sulle proprietà oggetto.

Una correzione rilevante riguarda il confine del pivot. La condizione originaria header > 17 è diventata header >= 17: il valore 17 deve infatti poter rappresentare una classe nominata con identificativo 0. Su questo punto la specifica è ambigua: le sezioni 1.3 e 4 trattano come tag ogni header minore o uguale al pivot, mentre la sezione 4.4 considera classe ogni header maggiore o uguale a 17. La modifica segue la sezione 4.4, la cui regola vale anche per i data range con il valore 5, e risolve un caso che poteva rimanere nascosto se il primo identificativo veniva assegnato all'IRI dell'ontologia.

Sono state aggiunte anche DataSomeValuesFrom, DataAllValuesFrom, DataHasValue e le cardinalità minima, massima ed esatta sulle data property; il dataset ne usa quattro (sezione 6.8). Nelle cardinalità sui dati, l'assenza del filler esplicito corrisponde al datatype rdfs:Literal. Il supporto comprende inoltre data range nominati, intersezioni, unioni, complementi, enumerazioni di literal e restrizioni di datatype con facet. Per DatatypeRestriction la specifica mostra soltanto l'array delle facet, mentre OWL 2 richiede anche il datatype di base: parser e renderer lo codificano come data range prima delle facet. Le strutture annidate vengono decodificate ricorsivamente e i relativi riferimenti sono risolti attraverso la tabella degli identificativi.

4.7 Assiomi

Oltre agli assiomi necessari per home e pizza, il parser gestisce DisjointUnion e SubObjectPropertyOf nella forma con catena di proprietà. Sono presenti le famiglie SubDataPropertyOf, EquivalentDataProperties, DisjointDataProperties, DataPropertyDomain, DataPropertyRange e FunctionalDataProperty, insieme a DatatypeDefinition e HasKey. Sono inoltre supportate le asserzioni negative su proprietà oggetto e dati e gli assiomi SubAnnotationPropertyOf, AnnotationPropertyDomain e AnnotationPropertyRange. Per DataPropertyDomain la specifica indica come dominio un data range, mentre in OWL 2 il dominio è una class expression: parser e renderer usano una class expression.

Restano disponibili le disgiunzioni di classi, le proprietà oggetto equivalenti, disgiunte e inverse, domain e range, le caratteristiche Functional, InverseFunctional, Reflexive, Irreflexive, Symmetric, Asymmetric e Transitive, e le relazioni SameIndividual e DifferentIndividuals. La costruzione degli assiomi è collegata a un metodo comune che applica le eventuali annotazioni lette dal frame. La sezione 6.7 indica quali costrutti implementati non sono verificati da alcun test.

4.8 Gestione degli errori

Il parser segnala esplicitamente versioni, frame, formati di literal e costrutti non supportati. I namespace e gli identificativi fuori intervallo producono OWLParserException; i test comprendono riferimenti non ridichiarati dopo Reset. Attraverso il manager OWLAPI gli errori sono esposti come OWLOntologyCreationException. Il rifiuto degli identificativi non dichiarati segue le istruzioni del corso: la specifica prevede invece che un identificativo sconosciuto sia decodificato come nuovo individuo anonimo (sezione 5.1, nota 4), comportamento non implementato.

La lettura dei VarInt strutturali è stata corretta per controllare il limite prima della conversione a int. Indici, conteggi e lunghezze non rappresentabili, codifiche oltre cinque byte e terminazioni premature vengono rifiutati esplicitamente. Le stringhe vengono lette progressivamente, evitando di allocare immediatamente una dimensione enorme dichiarata da un input troncato.

I campi di cardinalità usano un percorso distinto a 32 bit senza segno, memorizzato in long: includono infatti sia la cardinalità sia il bit del filler. La correzione simmetrica del renderer consente il round-trip fino a Integer.MAX_VALUE per le cardinalità OWLAPI.

4.9 Reset e maximum index size

La variante MIS_128 usa Reset per limitare la crescita delle tabelle di decodifica. Il parser interpreta separatamente i bit NS e ID: con NS=1 scarta i namespace non riservati; con ID=1 svuota gli identificativi; con entrambi impostati esegue entrambe le operazioni; con entrambi a zero non modifica le tabelle. Gli indici riservati dei namespace, da 0 a 4, restano disponibili. Le nuove dichiarazioni di namespace non riservati ripartono quindi da 5 e quelle degli identificativi da 0: la specifica parla di numerazione che riparte da zero, e l'interpretazione adottata tiene conto dei namespace impliciti. Gli 11 Reset del dataset che azzerano anche i namespace sono gestiti in questo modo e i relativi file superano il confronto della fase a.

Il Reset modifica soltanto le tabelle di decodifica: gli assiomi già costruiti e i prefissi del documento restano invariati. Il parser non applica una soglia propria ma reagisce ai Reset presenti nello stream; un riferimento a un namespace o a un identificativo non ridichiarato dopo l'azzeramento produce OWLParserException. Nei file MIS_128 del dataset compaiono 22.031 frame Reset, distribuiti in 94 file: 22.020 azzerano soltanto gli identificativi e 11 anche i namespace. Le combinazioni con il solo bit NS o senza bit non compaiono nel dataset e sono verificate dai test sintetici (sezione 6.3).

Sezione 5

Aggiornamento del Renderer

Il renderer è stato esteso per serializzare il contenuto necessario al dataset oltre alle ontologie home e pizza. Il flusso inizializza i namespace riservati, raccoglie IRI e individui anonimi, assegna gli identificativi, scrive versione e dichiarazioni, emette metadati e assiomi e termina con End. Le tabelle mantengono l'ordine di inserimento. La produzione resta nella codifica standard: non vengono emessi frame Reset né file MIS_128.

5.1 Raccolta degli identificativi

La raccolta preventiva comprende ontology IRI e version IRI, IRI degli imports, signature dell'ontologia corrente, individui anonimi referenziati, proprietà e valori delle annotazioni e soggetti delle AnnotationAssertion. Sono visitate anche le annotazioni degli assiomi e le loro sottoannotazioni, i data range annidati, i datatype e le facet, e gli IRI presenti esclusivamente nel domain o range di una annotation property. Non viene incorporata automaticamente la signature delle ontologie importate.

IRI e individui anonimi sono raccolti separatamente; gli indici vengono poi assegnati prima a tutti gli IRI e successivamente agli anonimi. L'emissione delle dichiarazioni segue lo stesso ordine, eliminando il rischio di disallineamento fra assegnazione e lettura quando le due categorie compaiono nella stessa ontologia. Sono raccolti anche gli individui anonimi usati come valori delle annotazioni.

5.2 Metadati e annotazioni

La serializzazione dei metadati segue la struttura v1. Il frame 0x06 include l'IRI dell'ontologia e, se presente, quello della versione, segnalato mediante il primo utility bit. Gli imports sono emessi come sequenza di identificativi nel frame 0x07. Le annotazioni dell'ontologia sono emesse nel frame 0x08, utilizzando la codifica dedicata di proprietà e valore. Le sottoannotazioni sono visitate e scritte ricorsivamente.

5.3 Namespace e prefissi

La gestione dei namespace ha richiesto una revisione specifica. Il renderer iniziale separava quelli con prefisso da quelli senza prefisso, rischiando di emettere le dichiarazioni in un

ordine diverso da quello usato per assegnare gli indici. Poiché il parser assegna gli indici in ordine di lettura, tale discrepanza poteva ricostruire IRI errati. La nuova emissione segue l'ordine della namespaceTable, mantenendo coerenti gli indici assegnati e quelli ricostruiti in lettura.

È stato inoltre introdotto il raggruppamento dei prefissi per namespace, così da conservare più alias associati allo stesso indirizzo. Per ogni namespace viene emessa prima la dichiarazione principale; gli alias aggiuntivi vengono scritti dopo tutte le dichiarazioni principali, evitando di spostare gli indici utilizzati dagli IRI. Il metodo collectPrefixNamespaces registra anche i namespace presenti esclusivamente nella mappa dei prefissi del document format. In questo modo un prefisso dichiarato nel documento può essere conservato anche se non compare nelle entità raccolte.

La scrittura normalizza i nomi dei prefissi secondo le convenzioni ProtocOWL. OWLAPI rappresenta i nomi con i due punti finali, mentre ProtocOWL usa il nome senza tale carattere. Prima di serializzare il prefisso il renderer rimuove quindi i due punti: dc: diventa dc e il prefisso di default : diventa una stringa di lunghezza zero. La correzione è necessaria anche se una successiva lettura OWLAPI potrebbe normalizzare nuovamente un prefisso scritto in modo improprio, nascondendo l'errore a un semplice test di round-trip.

5.4 Literal

La scrittura dei literal costruisce l'header con la formula type | (format << 2). Il renderer verifica prima la presenza di un language tag; in tal caso sceglie il tipo con lingua. Le stringhe senza lingua, compreso xsd:string, vengono codificate come plain. Gli altri literal mantengono il tipo typed e il riferimento al datatype.

Per i booleani viene utilizzato un solo byte di valore, 0 oppure 1. Per i datatype interi XML Schema riconosciuti, il renderer usa il formato 3 quando il valore è non negativo, incluso zero, e il formato 4 quando è negativo. In entrambi i casi scrive il modulo come unsigned VarInt, utilizzando BigInteger. Sono riconosciuti i 13 datatype interi di XML Schema [7]: integer, nonNegativeInteger, positiveInteger, nonPositiveInteger, negativeInteger, long, int, short, byte, unsignedLong, unsignedInt, unsignedShort e unsignedByte. Il riferimento al datatype continua a essere emesso dopo il valore.

Il formato compatto viene selezionato soltanto quando la conversione conserva la forma lessicale. Rappresentazioni non canoniche, come '+1', '001' o '-0', vengono scritte in forma

testuale mantenendo il datatype. Il valore intero non è limitato ai tipi Java int o long; rimangono distinti i limiti dei campi strutturali, come indici e lunghezze. Anche i decimali continuano a essere scritti come stringhe tipizzate: il renderer non produce i formati decimali compatti a virgola fissa e con esponente.

5.5 Assiomi

Tutti gli assiomi dell'ontologia passano attraverso un dispatch esplicito comune, dopo un ordinamento della sequenza. Il renderer conserva le declaration effettivamente presenti e gestisce anche DisjointUnion, catene di proprietà, assiomi sulle data property, DatatypeDefinition, HasKey, asserzioni negative e assiomi sulle annotation property, oltre alle famiglie già necessarie per home e pizza. Le AnnotationAssertion fanno parte dello stesso percorso di serializzazione.

Per ogni assioma annotato viene impostato il primo utility bit e scritta la sequenza delle annotazioni prima del payload. Nelle catene di proprietà il secondo bit è distinto da quello delle annotazioni. La conservazione riguarda anche declaration e annotation assertion annotate. Un tipo non riconosciuto genera un'eccezione esplicita invece di essere omesso, come verificato da un caso con una regola SWRL non supportata.

5.6 Espressioni di classe e cardinalità

Per le espressioni di classe vengono scritti i nuovi tag seguiti ricorsivamente dagli operandi. Le cardinalità sono codificate come (cardinalità << 1) | presenzaFiller. Quando il filler è owl:Thing il bit è zero e il filler non viene emesso; negli altri casi segue la relativa espressione di classe. Le proprietà oggetto dirette usano l'identificativo aumentato di uno; quelle inverse usano il tag zero seguito dall'identificativo della proprietà di base.

La serializzazione comprende anche le restrizioni e le cardinalità sulle data property, oltre ai data range nominati e complessi. Per le cardinalità sui dati il filler viene omesso soltanto quando coincide con il top datatype; rdf:PlainLiteral non viene trattato come equivalente a tale top. I data range sono scritti ricorsivamente, comprendendo combinazioni, enumerazioni di literal e restrizioni con facet. Le categorie non gestite generano un errore esplicito. Il campo cardinalità/filler viene costruito usando un intermedio long: anche la cardinalità massima del dominio int non subisce overflow nello spostamento di un bit.

5.7 Terminazione dello stream

Il renderer attuale emette un frame End esplicito dopo l'ultimo assioma. Il parser continua ad accettare anche la terminazione alla fine dello stream prevista dalla specifica. Poiché i binari standard del dataset terminano senza End, ogni file prodotto contiene un byte in più per questo frame.

5.8 Gestione dei costrutti non supportati

Il renderer non ignora più silenziosamente i tipi di assioma che non riconosce. Il dispatch termina con una IOException esplicita per gli assiomi non supportati; errori analoghi sono presenti per espressioni di classe, data range e valori di annotazione non gestiti. In questo modo un costrutto non supportato emerge come errore nell'analisi di copertura invece di produrre un file incompleto. Anche le mappe di prefissi non rappresentabili nella v1, cioè con alias dei namespace riservati o senza uno dei prefissi riservati, sono rifiutate prima di scrivere qualsiasi byte.

Sezione 6

Aggiornamento dei test e risultati

6.1 Risorse e organizzazione dei test

Le risorse home.owl/home.oprt e pizza.owl/pizza.oprt restano i riferimenti dei test iniziali. ProtocOWLTest utilizza un helper condiviso per confrontare ciascun binario con il Functional e per verificare il round-trip del renderer. I file prodotti sono build/out_home.oprt e build/out_pizza.oprt, relativamente alla directory lib. Poiché home.oprt usa la codifica precedente degli interi, i due binari di riferimento sono letti con la revisione LEGACY del parser (sezione 4.5); gli output del renderer e il dataset sono letti con la revisione corrente.

La nuova fase aggiunge DatasetRoundTripTest con un DataProvider TestNG che enumera esattamente 200 coppie: 100 ontologie per due varianti di input. Ogni coppia è un caso di test distinto. BenchmarkComplianceTest contiene regressioni sintetiche sul codec; CodecRobustnessTest comprende 15 casi su overflow e prefissi e IntegerEncodingTest 43 casi sulla rappresentazione numerica; BenchmarkResultsTest verifica il rifiuto di CSV incompleti o incoerenti. Cinque verifiche isolate in benchmark/test_run_benchmark.py controllano la pubblicazione dei risultati senza generare misure prestazionali.

I test sono separati in tre esecuzioni Gradle: test per i controlli rapidi, coverageTest per la fase 1 e datasetTest per la fase 2. Questa organizzazione consente di eseguire spesso la regressione veloce senza confonderla con la validazione dell'intero dataset.

6.2 Criteri di confronto

Il confronto condiviso verifica se le ontologie sono nominate e confronta ontology IRI e version IRI. Controlla inoltre le dichiarazioni di import, le mappe dei prefissi, le annotazioni dell'ontologia e l'insieme completo degli assiomi. Le annotazioni associate agli assiomi fanno parte degli oggetti confrontati; declaration e annotation assertion non vengono filtrate. L'ordine di serializzazione degli insiemi è irrilevante.

Quando gli assiomi differiscono, la diagnostica elenca quelli presenti soltanto nell'originale e quelli presenti soltanto nella versione riletta. Il controllo riguarda la struttura degli oggetti OWLAPI, non l'equivalenza logica ottenuta mediante ragionamento. Per gli individui anonimi viene disabilitata la rimappatura automatica degli identificativi durante il caricamento.

Il binario di riferimento pizza.oprt non dichiara l'alias pizza:, che in pizza.owl duplica il namespace del prefisso di default; su indicazione del docente, soltanto il confronto con questo file ne ammette l'assenza. Ogni altra differenza continua a determinare un fallimento. L'eccezione non si applica al dataset né al round-trip del renderer, nei quali le mappe dei prefissi restano confrontate esattamente.

6.3 Test negativi e regressioni mirate

Restano i casi che rifiutano una versione non supportata, un frame sconosciuto e uno stream troncato durante una dichiarazione. A questi si aggiungono controlli su namespace e identificativi non ridichiarati dopo Reset e su riferimenti invalidi nelle declaration. Sono verificate tutte le combinazioni dei bit NS/ID, la conservazione dei prefissi attraverso Reset e la possibilità che Reset sia il primo frame.

Le regressioni sintetiche comprendono assiomi prima omessi, assiomi annotati e annotazioni annidate, imports, ontology/version IRI, catene di proprietà, il rifiuto esplicito di SWRL, la ricostruzione della frazione invertita nei decimali e la conservazione di interi limite e forme lessicali non canoniche. Un controllo verifica inoltre che il passaggio attraverso Functional non aggiunga declaration mancanti. Il validatore CSV viene esercitato con righe mancanti, duplicate, vietate e dimensioni errate.

I test che passano attraverso il manager OWLAPI osservano OWLOntologyCreationException; alcuni casi invocano direttamente il parser e verificano OWLParserException. Le anomalie emerse durante la revisione sono state corrette e trasformate in regressioni in CodecRobustnessTest: riferimenti oltre i limiti dopo Reset, conteggi e lunghezze fuori intervallo, stringhe enormi troncate, cardinalità massime, prefissi espliciti e assenza di alias non presenti nell'originale. Sono verificati anche il confronto rigoroso e il rifiuto, prima dell'emissione, delle mappe non rappresentabili a causa dei namespace riservati.

I test numerici verificano inoltre il segno selezionato dai formati 3 e 4 e il modulo unsigned VarInt, la conservazione delle forme lessicali non canoniche e la gestione di input numerici troncati. Per i fixed point vengono controllati valori negativi con e senza parte intera, tra cui -0.5, -1.0 e -2.04, verificando la compensazione della componente SVarInt e la conservazione del segno.

6.4 Risultati della verifica

La verifica è stata eseguita come prerequisito della campagna di benchmark, con Gradle wrapper 9.2.0, target Java 21 e heap massimo dei test di 4 GiB.

- Test rapidi: 80/80 superati, senza skip (10 in ProtocOWLTest, 11 in BenchmarkComplianceTest, 15 in CodecRobustnessTest, 43 in IntegerEncodingTest e 1 in BenchmarkResultsTest).
- Copertura: 300/300 operazioni riuscite; tutte le 100 scritture comprendono rilettura e confronto con il Functional.
- Dataset: 200/200 casi superati, ciascuno con fasi a, b e c, senza skip o ontologie escluse.
- roundtrip_sizes.csv: 200 righe, con dimensioni uguali per le due varianti di ciascuna ontologia.
- Test Python dello script: cinque superati; sono prove di pubblicazione con processi simulati, distinte dalle misure reali.
- Benchmark: 500 misure completate e validate, con tutte le combinazioni previste e senza duplicati.

6.5 Dimensione dei file

La tabella seguente confronta le dimensioni in byte dei file di riferimento e di quelli prodotti dal renderer nei test:

Tabella 1 – Dimensioni in byte dei file di riferimento e dei file prodotti dal renderer

Ontologia	Sintassi funzionale	ProtocOWL di riferimento	ProtocOWL prodotto
home	13.836	1.788	1.788
pizza	69.789	16.181	16.244

Per home l'output ha la stessa dimensione del riferimento; per pizza è maggiore di 63 byte (0,39%). Rispetto alla sintassi funzionale, i file prodotti sono più piccoli dell'87,1% per home e del 76,7% per pizza.

Per home i due binari hanno la stessa dimensione ma contenuto diverso: l'uguaglianza byte a byte non è richiesta, perché ordine dei frame e scelte di rappresentazione possono differire pur ricostruendo gli stessi oggetti. Per il dataset, le dimensioni prodotte nella fase c sono registrate in roundtrip_sizes.csv e usate per validare il benchmark.

Sul dataset, i file prodotti nella fase c occupano in totale 31.745.616 byte, il 3,80% in più dei 30.584.816 byte dei binari standard forniti: sono più grandi in 61 ontologie, uguali in una e più piccoli in 38. Dal confronto frame per frame di alcuni file, le differenze dipendono da scelte del renderer: il frame End finale, assente nei binari standard; una NamespaceDeclaration per ogni namespace invece di un unico array; i nodeID degli individui anonimi scritti con il prefisso '_:' di OWLAPI, che in 00536.owl aggiungono 310.324 byte per 155.162 individui; l'ordine di assegnazione degli identificativi, che cambia la lunghezza dei VarInt e può aumentare o ridurre la dimensione.

6.6 Esecuzione della suite

Le istruzioni richiedono JDK 21 o successivo e l'utilizzo del wrapper Gradle. Dalla directory protocowl-owlapi, i controlli vanno eseguiti nell'ordine:

```
bash ./gradlew --no-daemon :lib:test
bash ./gradlew --no-daemon :lib:coverageTest
bash ./gradlew --no-daemon :lib:datasetTest
```

I report HTML sono generati nelle sottocartelle test, coverageTest e datasetTest di lib/build/reports/tests. Il dataset può essere configurato tramite DATASET_DIR o la proprietà Gradle datasetDir; METADATA_FILE o metadataFile consentono di specificare il manifest. Serve un JDK fra 21 e 25: il codice è compilato con target Java 21 e il wrapper Gradle 9.2.0 si esegue fino a Java 25.

6.7 Copertura dei test

La suite confronta anche gli imports e comprende casi di identificativo non dichiarato. Le regressioni sintetiche verificano inoltre assiomi annotati, annotazioni annidate e costrutti che non erano esercitati dai soli esempi home e pizza. L'analisi di copertura tenta tutte le 300 operazioni previste sui 100 input, senza ridurre il campione in funzione dell'esito.

La lettura è verificata su binari prodotti da un encoder indipendente: home.oprt, pizza.oprt e i 200 file del dataset. Per i costrutti assenti da questi file, il renderer è verificato soltanto rileggendo i propri output con il parser dello stesso progetto, quindi un errore commesso in modo simmetrico da entrambi non sarebbe rilevato. Alcuni costrutti implementati non compaiono né nei file di riferimento né nei test sintetici: ObjectHasSelf, ObjectInverseOf, DataSomeValuesFrom, i data range complessi (intersezione, unione, complemento, enumerazione e restrizione con facet), SameIndividual, EquivalentObjectProperties,

DisjointObjectProperties e le caratteristiche Reflexive, Irreflexive, Symmetric e Asymmetric.

6.8 Analisi di copertura sul dataset

CoverageAnalyzer usa lo stesso manifest impiegato nei test di round-trip e nel benchmark. Per ogni ontologia esegue tre operazioni: parsing del binario standard, parsing del binario MIS_128 e serializzazione del Functional con il renderer standard. Quest'ultima operazione comprende anche la rilettura dell'output e il confronto con l'ontologia caricata dal Functional.

Il rapporto lib/coverage_report.csv contiene le colonne Ontology, Input, Operazione, Esito, Costrutto e Messaggio. Gli errori vengono raggruppati per categoria di costrutto per facilitare la diagnosi; un errore determina il fallimento del controllo, senza eliminare l'ontologia dal dataset. La verifica corrente registra 300 operazioni riuscite. Le due operazioni di parsing non confrontano da sole il contenuto con il Functional: tale responsabilità appartiene alla fase successiva.

Oltre ai costrutti di pizza.owl, il dataset contiene (tra parentesi il numero di ontologie): dichiarazioni di data property (10), ObjectMaxCardinality (6), ObjectExactCardinality (12), DataHasValue (8), DataMinCardinality (2), DataAllValuesFrom (1), DataExactCardinality (1) e individui anonimi (20). I binari usano inoltre literal tipizzati nei formati 3 (1 ontologia), 5 (fixed point, 5) e 6 (E-notation, 4) e, nei file MIS_128, il frame Reset (94 file). Gli altri costrutti aggiunti nelle sezioni 4 e 5, come DisjointUnion, catene di proprietà, assiomi sulle data property e sulle annotation property, DatatypeDefinition, HasKey, asserzioni negative e data range complessi, non compaiono nel dataset e sono verificati, quando lo sono, dai test sintetici (sezioni 6.3 e 6.7).

6.9 Round-trip completo e conservazione strutturale

DatasetRoundTripTest associa a ogni ontologia due casi indipendenti, uno per il binario standard e uno per MIS_128. Nella fase a carica il binario e lo confronta con il Functional di riferimento. Nella fase b serializza in Functional l'ontologia ottenuta in a, la rilegge e ripete il confronto con l'originale. Nella fase c serializza il risultato di b in ProtocOWL standard, lo rilegge e confronta nuovamente con lo stesso originale. La produzione ProtocOWL è sempre senza Reset.

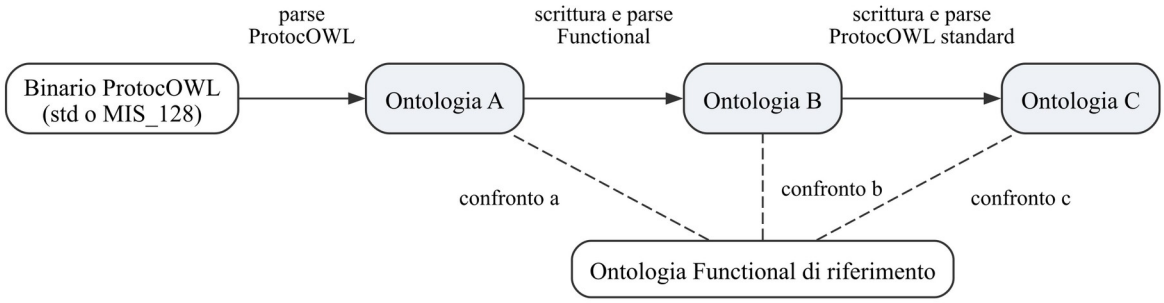


Figura 1 – Percorso verificato nella fase 2 per ogni coppia (ontologia, variante): le ontologie A, B e C sono confrontate con l'ontologia Functional di riferimento

Per evitare che il passaggio Functional introduca informazioni non presenti, viene usato il renderer OWLAPI con aggiunta automatica delle declaration disabilitata e con una mappa dei prefissi esplicita. Gli identificativi degli individui anonimi vengono preservati durante il caricamento.

Gli errori riportano ontologia, variante e fase e includono le differenze fra gli insiemi di assiomi. Per ogni caso che completa c viene registrata la dimensione dell'output ProtocOWL in roundtrip_sizes.csv, con colonne Ontology, InputVariant e OutputSizeBytes. Tutti i 200 casi completano le fasi a, b e c; roundtrip_sizes.csv contiene quindi 200 righe dati. Per ciascuna ontologia, le due varianti di input producono output standard della stessa dimensione.

Sezione 7

Benchmark: metodologia e stato della validazione

7.1 Dataset e combinazioni di misura

Il campione comprende 100 ontologie ORE 2014 [2], ciascuna disponibile in Functional Syntax, ProtocOWL standard e ProtocOWL MIS_128. Il Functional è il riferimento per il contenuto. Nella distribuzione locale i file sono organizzati in dataset_onto/functional, dataset_onto/protocowl/std e dataset_onto/protocowl/MIS_128; la risoluzione dei percorsi accetta anche la cartella protocowl/standard prevista dalle istruzioni.

DatasetFiles legge un elenco esplicito di 100 ontologie: il file indicato da METADATA_FILE, se impostato, altrimenti metadata.csv nella radice del dataset o, in sua assenza, benchmark/metadata.csv. Quest'ultimo contiene le 100 righe del metadata ORE corrispondenti ai file forniti, quindi l'elenco coincide con l'intero dataset. Duplicati, cardinalità diversa da 100 e input mancanti vengono segnalati.

Le combinazioni previste sono parse e render per Functional, parse e render per ProtocOWL e soltanto parse per ProtocOWL_128. Una campagna completa deve quindi produrre 500 righe dati: 200 Functional, 200 ProtocOWL e 100 ProtocOWL_128. Una richiesta esplicita di render per ProtocOWL_128 viene rifiutata.

7.2 Misura di tempo e memoria

run_benchmark.sh esegue prima i test rapidi, la copertura e i round-trip. Solo dopo il loro completamento avvia le misure, ciascuna in una JVM separata sotto GNU time [8] con opzione -v. BenchmarkTask usa System.nanoTime per misurare l'operazione richiesta. Nel parsing l'inizializzazione del manager precede il timer; nel rendering il caricamento dell'ontologia dal file di input dello stesso formato, la scelta del formato e la copia dei prefissi precedono la misura della serializzazione su un file temporaneo. Il tempo include il lavoro di ingresso/uscita interno all'operazione misurata.

L'MRSS fornito da GNU time è il massimo resident set size dell'intero processo Java, espresso in KB: comprende JVM, strutture OWLAPI e, nel render, il caricamento preliminare dell'ontologia, escluso invece dal tempo misurato.

Lo script produce una sola misura per ciascuna tripla task/formato/ontologia, senza warm-up né ripetizioni: ogni tempo comprende caricamento delle classi e compilazione JIT di una JVM appena avviata, e la variabilità fra esecuzioni ripetute non è stimabile.

Nel parse BenchmarkTask registra nel manager OWLAPI soltanto il parser del formato misurato. Senza questa scelta OWLAPI proverebbe i parser disponibili uno dopo l'altro finché uno riesce: per un file Functional il tempo comprenderebbe anche i tentativi falliti dei parser ProtocOWL, RDF/XML e OWL/XML, mentre un file ProtocOWL verrebbe letto al primo tentativo.

7.3 CSV e validazione delle metriche

Il file previsto è benchmark_results.csv e contiene l'intestazione:

```
Task,Format,Ontology,TimeMs,InputSizeBytes,OutputSizeBytes,MRSS_KB,
CompressionRatioVsProtocOWL,SpaceSavingVsProtocOWL
```

TimeMs è il tempo dell'operazione; InputSizeBytes è la dimensione del file di input nel formato considerato; OutputSizeBytes è zero nel parsing e misura il file prodotto nel rendering. Le metriche dimensionali usano sempre come riferimento il binario standard della stessa ontologia:

```
CompressionRatioVsProtocOWL = size(formato) / size(ProtocOWL
standard)
SpaceSavingVsProtocOWL = [size(formato) - size(ProtocOWL
standard)] / size(formato)
```

Queste formule usano i file di input del dataset anche per le righe render, non la dimensione dell'output appena prodotto. Per ProtocOWL standard il rapporto vale 1 e il risparmio vale 0. Il risparmio indica la riduzione ottenibile usando lo standard rispetto al formato della riga; un valore negativo significa che il file del formato considerato è più piccolo dello standard.

BenchmarkResults controlla il numero di righe, tutte le combinazioni attese, l'assenza di duplicati e di render MIS_128, la validità delle metriche numeriche e le dimensioni degli input. Confronta inoltre OutputSizeBytes del render ProtocOWL con la dimensione registrata nella fase c per la variante standard. Le combinazioni mancanti o le incoerenze producono un errore, invece di essere trattate come dati da scartare.

7.4 Ambiente ed esecuzione riproducibile

La campagna richiede Bash e GNU time con supporto all'opzione -v. Su Linux o WSL si può verificare /usr/bin/time -v true. Su macOS il comando BSD /usr/bin/time non offre la

stessa interfaccia; lo script consente di usare gtime attraverso TIME_CMD. Le opzioni delle JVM di misura sono configurabili mediante BENCH_JAVA_OPTS; quelle usate nella campagna sono riportate più avanti.

Lo script prepara un rapporto dell'ambiente con data, sistema operativo, informazioni hardware disponibili, versione del JDK, opzioni JVM, comando time, revisione Git e manifest dei file. CSV e ambiente vengono scritti in file temporanei e sostituiscono benchmark_results.csv e benchmark_environment.txt soltanto dopo il superamento dei prerequisiti e della validazione finale. Un fallimento non cancella gli ultimi risultati riusciti. Il comando previsto, dalla radice protocowl-owlapi, è: bash ./run_benchmark.sh

La campagna è stata eseguita su un MacBook Pro (identificativo Mac15,6) con Apple M3 Pro, 12 CPU logiche e 18 GiB di RAM, macOS 26.6.2 (kernel Darwin 25.6.0, arm64), Gradle wrapper 9.2.0 e target Java 21. Le JVM di misura usano Oracle JDK 26.0.2.1 (HotSpot) con -Xms256m -Xmx4g; i test preliminari hanno heap massimo di 4 GiB. L'MRSS è rilevato con GNU time 1.10, che su macOS converte ru_maxrss da byte a KB.

L'esecuzione ha usato DATASET_DIR sul dataset, METADATA_FILE sull'elenco incluso e TIME_CMD su GNU time; benchmark_environment.txt registra i percorsi completi. Dopo il superamento dei prerequisiti sono state eseguite 500 misure in JVM separate.

7.5 Stato della validazione e anomalie nei dati

La validazione è completa: 300 operazioni di copertura e 200 round-trip superati, senza casi esclusi (sezione 6.4). I materiali forniti presentano però alcune anomalie, gestite senza modificarli: home.oprt usa la codifica degli interi della revisione precedente della specifica ed è letto con la revisione LEGACY del parser (sezione 4.5); pizza.oprt non dichiara l'alias pizza:, assenza ammessa su indicazione del docente nel solo confronto con quel file (sezione 6.2); la codifica della parte intera dei fixed point negativi non è descritta dalla specifica ed è stata chiarita dal docente.

Nella specifica sono emersi inoltre alcuni punti ambigui o in contrasto con OWL 2, risolti come descritto nelle sezioni 4.6, 4.7 e 4.8: il confine del pivot, la DatatypeRestriction senza datatype di base, il dominio di DataPropertyDomain indicato come data range e la decodifica degli identificativi sconosciuti come nuovi individui anonimi. Poiché il dataset non contiene

DatatypeRestriction né DataPropertyDomain, le scelte relative a questi due costrutti non sono confrontate con file di riferimento.

La campagna finale contiene 500 misure reali validate, senza combinazioni mancanti, duplicate o render MIS_128. Le dimensioni degli input corrispondono ai file del dataset e quelle degli output ProtocOWL coincidono con le dimensioni registrate nella fase c del round-trip.

7.6 Risultati della campagna

La tabella riporta, per ogni combinazione, la mediana sulle 100 ontologie, ciascuna misurata una sola volta: la mediana descrive quindi il campione di ontologie, non la dispersione di esecuzioni ripetute.

Tabella 2 – Mediane di tempo e MRSS sulle 100 ontologie del dataset

Task	Formato	N	Tempo mediano (ms)	MRSS mediano (KB)
parse	Functional	100	119,521	126.104
parse	ProtocOWL	100	99,311	118.216
parse	ProtocOWL_128	100	114,798	130.920
render	Functional	100	90,788	145.496
render	ProtocOWL	100	117,620	150.656

Le dimensioni complessive degli input, contando una volta ciascuna ontologia, sono: Functional 293.026.071 byte, ProtocOWL standard 30.584.816 byte e MIS_128 47.065.571 byte. Sui totali, lo standard occupa l'89,56% in meno del Functional e MIS_128 il 53,89% in più dello standard. Questi rapporti aggregati sono calcolati sui totali e non sono la media dei rapporti per ontologia del CSV. Per singola ontologia, il rapporto fra Functional e standard ha mediana 11,80 (da 3,23 a 31,06); MIS_128 è più grande dello standard in 95 ontologie, con rapporto mediano 1,66, e più piccolo in 5, verosimilmente perché dopo ogni Reset gli indici restano sotto 128 e occupano un solo byte, compromesso previsto per questa variante.

Nel parse, ProtocOWL standard è più rapido del Functional in 98 ontologie su 100 (mediana 99,311 contro 119,521 ms, -16,9%) e in tutte le 50 con il file Functional più grande; ha inoltre MRSS inferiore in 90 casi (mediana -6,3%). La variante MIS_128 è più lenta dello standard in 90 ontologie (mediana +15,6%) e ha MRSS maggiore in 86 (mediana +10,7%): i suoi file sono più grandi e il rapporto fra i tempi cresce con quello fra le dimensioni (correlazione di Spearman 0,60). In questo campione l'azzeramento delle tabelle non riduce

l'MRSS dell'intero processo; un'ipotesi è che questo sia dominato da JVM e strutture OWLAPI, ma la memoria del solo codec non è stata misurata.

Nel render, ProtocOWL è più lento del Functional in 76 ontologie su 100 (mediana +29,6%; 49,3 s contro 16,9 s in totale) e il divario cresce con la dimensione: nelle 50 ontologie con il file Functional più grande è più lento in 48 casi, fino a 6 volte. Un'ipotesi da verificare con un profiler è il costo delle due visite complete degli assiomi eseguite dal renderer, ciascuna preceduta da un ordinamento: una per raccogliere gli identificativi e una per scrivere gli assiomi. In tutte le combinazioni il tempo cresce con la dimensione dell'input (correlazione di Spearman fra 0,97 e 0,99), come mostra la Figura 2.

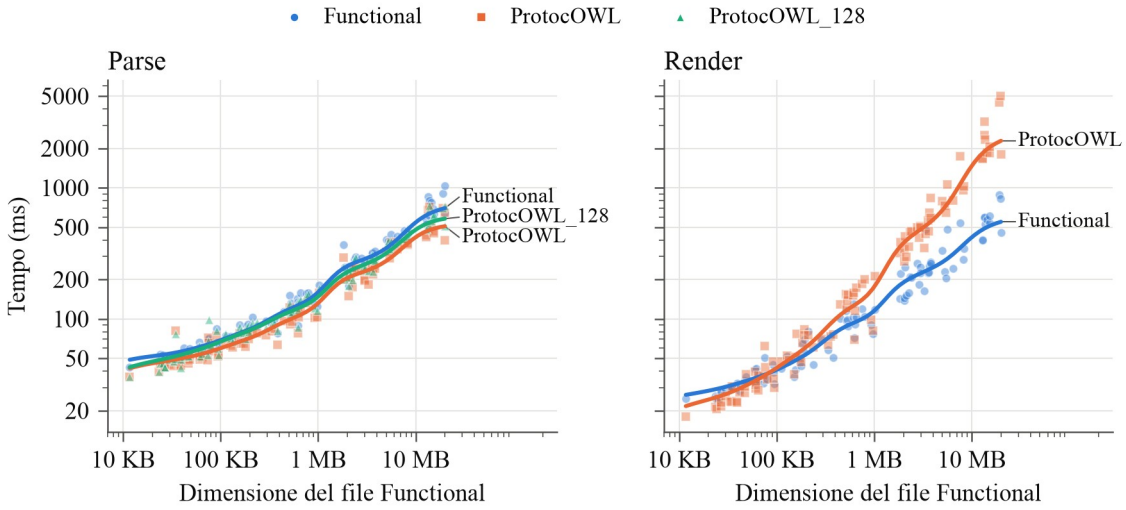


Figura 2 – Tempo di parse e di render di ogni ontologia in funzione della dimensione del file Functional, in scala logaritmica; le curve indicano l'andamento di ciascun formato

Sezione 8

Conclusioni e sviluppi futuri

Il progetto è passato dal supporto iniziale di home e pizza a una validazione strutturata su 100 ontologie e due varianti ProtocOWL. Le estensioni comprendono Reset in lettura, restrizioni e cardinalità sulle data property, data range, ulteriori famiglie di assiomi, annotazioni degli assiomi e una raccolta più completa dei riferimenti. Il renderer standard segnala i costrutti non supportati e termina con End.

Una revisione successiva del codice ha corretto l'overflow dei VarInt strutturali e delle cardinalità compattate, introdotto una lettura progressiva delle stringhe e reso esatto il confronto dei prefissi. Il parser non ricostruisce più alias assenti dallo stream; il renderer rifiuta esplicitamente le mappe che non possono essere conservate rispettando i namespace riservati della v1. I risultati del benchmark sono pubblicati soltanto dopo la validazione.

La validazione registra 80 test rapidi superati, 300 operazioni di copertura riuscite e 200 round-trip completi, senza casi saltati o ontologie escluse. Sono superati anche i cinque test dello script. Le anomalie dei file di riferimento e i punti ambigui della specifica sono documentati nella sezione 7.5.

Nella campagna finale, con 500 misure validate, i binari ProtocOWL standard occupano in totale l'89,56% in meno dei file Functional e la variante MIS_128 il 53,89% in più dello standard. Il parse ProtocOWL è più rapido del Functional in 98 ontologie su 100, mentre il render ProtocOWL è più lento in 76 ontologie su 100.

8.1 Difficoltà riscontrate

La prima difficoltà è stata la rimappatura dei codici. La nuova specifica riutilizza valori della precedente implementazione con significati diversi, per cui una modifica incompleta può produrre un errore di interpretazione senza segnalare immediatamente una violazione sintattica. È stato necessario ragionare sull'intera struttura dei frame, compresi gli utility bit, anziché limitarsi a sostituire singoli numeri.

Un problema analogo riguarda il pivot encoding. Il valore di confine deve essere interpretato in modo coerente con l'indicizzazione che parte da zero. La correzione da header > 17 a

header >= 17 mostra come un solo operatore possa distinguere un riferimento valido a una classe da un costrutto non riconosciuto. Anche per le cardinalità è stato necessario coordinare il valore numerico, il bit che segnala il filler e la presenza effettiva dei dati nello stream.

I literal hanno richiesto di separare tre aspetti: tipo OWL, formato binario e valore lessicale. Un intero compatto non perde il datatype; una stringa xsd:string deve essere mappata correttamente alla forma plain; un literal con lingua deve mantenere il language tag. In particolare, il controllo prioritario di hasLang evita che le etichette multilingua di pizza siano trattate come generici literal tipizzati.

La raccolta degli identificativi è risultata più ampia della sola signature. Il version IRI e gli IRI usati nelle annotazioni devono essere registrati prima della scrittura anche quando non identificano entità logiche dell'ontologia. La stessa esigenza vale per i soggetti delle AnnotationAssertion. Questa differenza tra entità e riferimenti serializzabili è stata affrontata estendendo la visita del renderer.

La gestione dei prefissi ha richiesto particolare attenzione perché i loro nomi sono informazioni di serializzazione, mentre gli IRI determinano i riferimenti delle entità. Due mappe possono contenere gli stessi namespace ma alias diversi. Il caso pizza, che usa sia il prefisso di default sia pizza:, ha evidenziato la necessità di conservare più alias e di mantenere stabile l'ordine delle dichiarazioni. La normalizzazione dei due punti e la raccolta dei namespace non utilizzati dalla signature hanno completato questo intervento.

Anche i test hanno evidenziato la differenza tra un errore interno al parser e l'eccezione esposta dal manager OWLAPI. La correzione delle aspettative nei test negativi ha allineato la verifica al punto di ingresso realmente utilizzato.

L'estensione al dataset ha reso evidente che un numero uguale di assiomi o la semplice rileggibilità di un file non bastano a dimostrare la conservazione. È stato necessario verificare l'intera catena a/b/c e confrontare assiomi, annotazioni, identità e imports. Il passaggio intermedio attraverso Functional ha inoltre richiesto di disabilitare l'aggiunta automatica di declaration e di controllare esplicitamente i prefissi.

Il Reset ha introdotto la necessità di separare le tabelle di decodifica dai metadati del documento. Azzerare gli indici non deve cancellare i prefissi né gli assiomi già costruiti. Per gli assiomi annotati e le catene di proprietà è stato necessario distinguere utility bit con funzioni diverse e correggere la codifica della proprietà nelle annotazioni annidate.

La lettura dei decimali compatti ha richiesto una verifica fino al livello dei byte. La frazione è memorizzata con le cifre invertite; per i fixed point negativi occorre inoltre applicare alla parte intera la compensazione. Mantenere separato il segno durante la ricostruzione dello zero consente di distinguere -0.5 da 0.5.

La verifica dei prefissi ha mostrato la differenza fra conservare un alias e ricostruirlo per convenzione. La seconda operazione mascherava il prefisso mancante nel binario di pizza e poteva aggiungere alias ad altri documenti. La rimozione dell'euristica e delle normalizzazioni nei test rende ora visibili le differenze reali. Per gli alias dei namespace riservati è stato necessario distinguere un limite del formato da una perdita silenziosa del renderer.

8.2 Sviluppi futuri

Analisi delle prestazioni del renderer: Il benchmark ha mostrato che il renderer ProtocOWL è più lento di quello Functional in 76 ontologie su 100, con differenze particolarmente evidenti sulle ontologie più grandi. Uno sviluppo futuro consiste nell'analizzarne l'esecuzione attraverso un profiler, uno strumento che permette di individuare le parti del codice che richiedono più tempo. In particolare, sarebbe utile valutare il costo delle due visite degli assiomi, ciascuna preceduta da un ordinamento: la prima raccoglie gli identificativi, mentre la seconda scrive il contenuto binario. Questa analisi permetterebbe di scegliere ottimizzazioni sulla base di misure concrete.

Ripetizione delle misure del benchmark: Attualmente viene effettuata una sola misura per ogni combinazione di ontologia, formato e operazione. Ripetere ciascuna misura permetterebbe di valutare quanto i risultati variano tra le esecuzioni e di confrontare valori più rappresentativi, come la mediana. Si potrebbe inoltre introdurre una fase iniziale di warm-up, con esecuzioni preliminari che consentano alla JVM di caricare le classi e ottimizzare il codice prima delle misure. Questo renderebbe più solido il confronto delle prestazioni a regime.

Supporto ai frame Add e Remove: Un'ulteriore estensione riguarda i frame Add e Remove, attualmente non supportati, per gestire aggiornamenti incrementali dell'ontologia. Per esempio, quando cambia un'informazione rilevata da un sensore, si potrebbe trasmettere la rimozione del vecchio assioma e l'aggiunta di quello aggiornato, senza inviare nuovamente l'intera ontologia. L'implementazione richiederebbe di applicare correttamente queste modifiche al modello OWLAPI e verificarne il risultato con test dedicati.

Bibliografia

[1] I. Bilenchi and V. Di Ceglie, “ProtocOWL specification, version 1.0,” SisInf Lab, Politecnico di Bari, Bari, Italy, Sep. 2026.

[2] N. Matentzoglu and B. Parsia, Jul. 7, 2014, “ORE 2014 Reasoner Competition Dataset,” Zenodo, doi: 10.5281/zenodo.10791.

[3] M. Ruta, F. Scioscia, V. Di Ceglie, I. Bilenchi, F. Gramegna, and E. Di Sciascio, “A format and protocol for machine-to-machine OWL data streams,” in *Proc. 10th Int. Workshop Adv. Sensors Interfaces (IWASI)*, Manfredonia, Italy, Jul. 3–4, 2025, pp. 1–6, doi: 10.1109/IWASI66786.2025.11121935.

[4] M. Horridge and S. Bechhofer, “The OWL API: A Java API for OWL ontologies,” *Semantic Web*, vol. 2, no. 1, pp. 11–21, 2011, doi: 10.3233/SW-2011-0025.

[5] B. Motik, P. F. Patel-Schneider, and B. Parsia, Eds. *OWL 2 Web Ontology Language Structural Specification and Functional-Style Syntax (Second Edition)*, W3C Recommendation, World Wide Web Consortium, Dec. 11, 2012. [Online]. Available: https://www.w3.org/TR/2012/REC-owl2-syntax-20121211/

[6] M. Horridge, *A Practical Guide to Building OWL Ontologies Using Protégé 4 and CO-ODE Tools*, ed. 1.3. Manchester, U.K.: Univ. of Manchester, 2011.

[7] D. Peterson, S. Gao, A. Malhotra, C. M. Sperberg-McQueen, and H. S. Thompson, Eds. *W3C XML Schema Definition Language (XSD) 1.1 Part 2: Datatypes*, W3C Recommendation, World Wide Web Consortium, Apr. 5, 2012. [Online]. Available: https://www.w3.org/TR/2012/REC-xmlschema11-2-20120405/

[8] D. Keppel, D. MacKenzie, A. H. Juul, and F. Pinard. *GNU Time*. (Apr. 14, 2026). GNU Project. (Version 1.10). Free Software Foundation. [Online]. Available: https://www.gnu.org/software/time/